

NexStar Finance: Enterprise Data Management Architecture and Implementation Plan

Donald Pellerin

Southern New Hampshire University

DAT 515 – Enterprise Data Management

Instructor: Ricky Sethi

January 18, 2026

Executive Summary

NexStar Finance is a fintech company offering digital banking, peer-to-peer transfers, and cryptocurrency services across the United States, Latin America, and parts of Africa. Rapid global expansion since 2017 has created a fragmented and inconsistent data environment marked by disconnected systems, legacy dependencies, and governance gaps. Customer, vendor, and transaction data are distributed across different cloud platforms and additional systems inherited from acquisitions. This has led to limited cross-regional reporting, operational efficiency, and regulatory compliance. As competitors with unified data architectures gain market share, NexStar's own share has declined from 16.2% in 2021 to 12.8% in Q1 of this year.

Leadership has mandated a development of a scalable enterprise data strategy, consolidate systems, and achieve compliance with global data standards within 18 months. This report evaluates NexStar's current enterprise data architecture, identifies critical gaps relative to industry frameworks, and proposes foundational improvements to support a unified, governed, and analytics-ready data environment.

Current State Assessment

NexStar's enterprise architecture consists of legacy relational databases and multiple independent cloud platforms from acquisitions. These operate with inconsistent data models, duplicated customer records, and uneven security practices and controls. Customer, vendor, and transaction data remain siloed, and the organization lacks a master customer record, standardized metadata, and a unified data catalog. The absence of a single source of truth limits reporting accuracy, fraud detection, and the ability to perform reliable cross-regional analytics.

Although newer applications use document-based or NoSQL-style structures, these tools are not integrated into a broader enterprise architecture. The organization lacks standardized pipelines and orchestration layers, resulting in redundant processes and fragmented reporting across business units. Consequently, the current architecture does not effectively support rapid growth.

When evaluated against the DAMA-DMBOK2 framework, NexStar is missing several core data enterprise management components. The company lacks a formal data governance program, defined stewardship roles, consistent data policies, and master data management practices. Metadata management is limited, restricting data lineage and data quality oversight. NexStar also lacks centralized analytics infrastructure, such as an enterprise data warehouse, data lake, or scalable platform for integrating structured and unstructured data.

Several existing tools are present but underperforming. Legacy SQL systems cannot efficiently manage high-volume, multi-regional workloads, cloud platforms operate without shared governance, and reporting tools fail to deliver consolidated enterprise insights. While NoSQL-style capabilities exist, NexStar is not leveraging document flexibility, embedded queries, or sharding to support global operations.

Key components missing entirely include formal data governance, MDM for customer and vendor records, standardized metadata management, a unified data model, centralized analytics platforms, and defined stewardship roles for data quality. Without these elements, NexStar cannot ensure consistency, security, or reliable cross-regional analytics.

Operational impacts are significant. In the past, acquisition-inherited systems were never integrated, leading to redundant workflows and conflicting datasets. Currently, business teams

lack access to reliable cross-regional insights, slowing decision-making and limiting fraud detection and customer analysis. Looking ahead, NexStar will struggle to scale, adopt advanced analytics or AI capabilities, or meet emerging global regulatory requirements without a unified data architecture. These limitations increase operating costs, duplicate effort, and weaken strategic decision-making, underscoring the need for immediate architectural and governance improvements.

Data Model Design and Implementation

Nexstar Finance requires a foundational data model capable of supporting core financial analytics, including account balances, deposits, withdrawals, and transaction activity. To support these requirements, a relational schema was designed using three primary entities: Customers, Accounts, and Transactions. This structure captures customer identity, account ownership, and financial activity, enabling analysis of customer behavior, account performance, and transaction volume across regions.

The Customers table stores a single record per customer and includes attributes for customer identification, contact information, geographic region, and registration date. The Accounts table stores one record per financial account, including account type, current balance, and currency, and links each account to a customer through a foreign key relationship. The Transactions table records individual financial events, including deposits, withdrawals, purchases, and transfers, and links each transaction to a specific account.

Primary keys were assigned to each table to uniquely identify records, and foreign key constraints were implemented to enforce referential integrity between Customers and Accounts and between Accounts and Transactions. These relationships follow standard one-to-many

patterns, where a customer may own multiple accounts and each account may contain multiple transactions. This structure supports core banking analytics and provides a scalable foundation for enterprise reporting.

A synthetic sample dataset was created to populate each table with ten representative records. The sample data supports analytics such as customer counts by region, total and average account balances, transaction volumes, and identification of high-activity or high-value customers. Appendix A contains the complete sample datasets and screenshots documenting successful table creation, data population, and query execution.

SQL Query Design and Business Relevance

Ten SQL queries were developed to demonstrate core enterprise reporting and analytics capabilities using joins, filtering, sorting, and aggregate functions. These queries support common financial reporting needs, including customer segmentation, geographic analysis, transaction monitoring, and exception reporting.

The queries include calculating account counts per customer, aggregating balances by region, identifying high-balance accounts, generating transaction audit trails, comparing deposits and withdrawals by account, identifying high-activity customers, detecting large transactions for risk review, determining transaction date ranges, filtering recent transactions, and analyzing regional customer balances. Together, these queries validate the schema design, demonstrate relational integrity across all three tables, and illustrate how the data model supports operational monitoring, compliance reporting, and strategic decision-making.

Technology Recommendations

To address NexStar's fragmented data environment and support scalable analytics, two core technologies are recommended. First, a cloud-based data warehouse platform such as Snowflake or Azure Synapse would provide centralized storage and scalable processing for transaction and customer data. This platform would reduce analytics workloads on operational systems and enable consolidated enterprise reporting. Potential challenges include ongoing infrastructure costs and the need for staff training to manage the platform effectively.

Second, a business intelligence and visualization platform such as Microsoft Power BI would enable standardized dashboards and real-time reporting for stakeholders. These tools support customer segmentation, account monitoring, and transaction analysis while improving decision-making speed. Challenges include licensing costs and the need for strong governance controls to prevent inconsistent or redundant reporting.

Data Management Improvements

To improve structured data management, NexStar should implement formal data governance practices aligned with DAMA-DMBOK principles. These include enforcing data quality rules, maintaining a centralized data dictionary, defining standardized business metrics, and assigning data ownership and stewardship roles for core entities. These practices ensure data consistency, accuracy, and reliability across systems.

For unstructured data, NexStar should implement a centralized document management strategy with classification standards and role-based access controls. Linking unstructured documents to structured customer records through unique identifiers improves traceability while maintaining security and regulatory compliance.

Operational Impact and Ethical Considerations

The recommended technologies and governance improvements will centralize data, reduce manual reporting, and enable faster, more accurate enterprise analytics. Unified data sources improve fraud detection, regulatory reporting, and customer insight while supporting scalable growth.

Ethically, NexStar must enforce strong access controls, encryption, and privacy protections to safeguard sensitive financial information. Governance policies should ensure transparency, prevent unauthorized access, and support compliance with global data protection regulations. These controls are essential for maintaining customer trust and responsible use of data in analytics and automated decision-making.

Conclusion

Nexstar Finance's rapid growth and acquisition-driven expansion have created a fragmented data environment that limits analytics, governance, and regulatory readiness. This report evaluated NexStar's current enterprise data architecture against industry standards, designed a foundational relational data model, and proposed governance and technology improvements to support a unified, scalable, analytics-read platform.

By implementing a centralized cloud data warehouse, enterprise business tools, and formal data governance practices aligned with DAMA-DMBOK principles, NexStar can eliminate data silos, improve reporting accuracy, and strengthen regulatory compliance. The proposed data model and analytics framework provide a reliable foundation for fraud detection, customer insight, and regional performance analysis while supporting future adoption of advanced analytics and artificial intelligence capabilities.

Over time, these recommendations position NexStar to expand centralized master data management, integrate additional unstructured data sources, and implement automated data quality monitoring and metadata management tools. Continued investment in governance, security controls, and scalable analytics platforms will enable sustainable growth, improved decision-making, and long-term competitiveness in an increasingly data-driven financial services market.

Appendix A – Database Schema, Sample Data, and SQL Artifacts

Appendix A contains technical artifacts supporting the proposed enterprise data model, including schema specifications, sample datasets, SQL queries, and screenshots documenting successful implementation and query execution.

Schema Specification

Customers

- CustomerID (INT, Primary Key)
- FullName (VARCHAR(100))
- Email (VARCHAR(50))
- Region (VARCHAR(50))
- RegistrationDate (DATE)

Accounts

- AccountID (INT, Primary Key)
- CustomerID (INT, Foreign Key → Customers.CustomerID)
- AccountType (VARCHAR(30))
- Balance (DECIMAL(12,2))
- Currency (CHAR(3))

Transactions

- TransactionID (INT, Primary Key)

- AccountID (INT, Foreign Key $\rightarrow$ Accounts.AccountID)
- TransactionType (VARCHAR(30))
- Amount (DECIMAL(12,2))
- TransactionDate (DATETIME)

Primary Keys

- Customers.CustomerID
- Accounts.AccountID
- Transactions.TransactionID

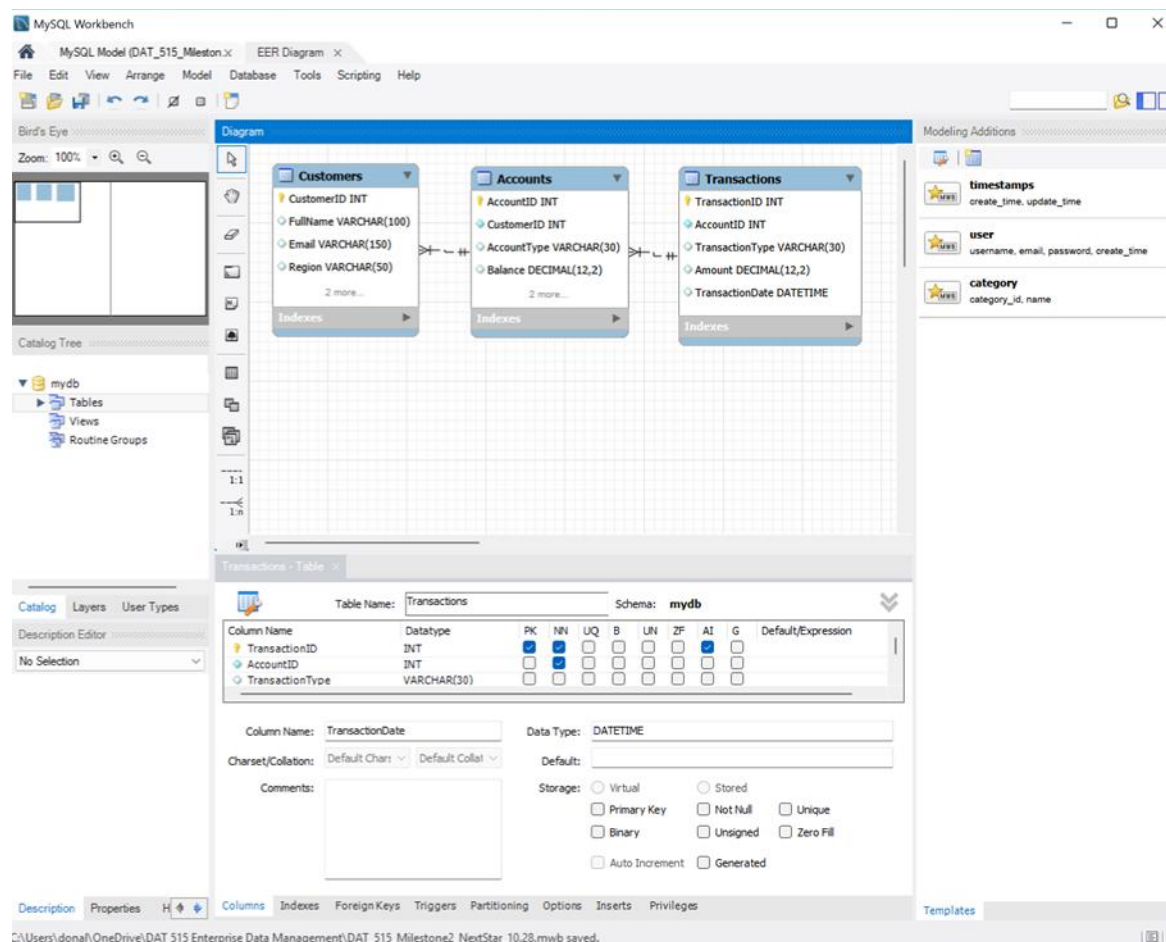
Foreign Key Relationships

- Accounts.CustomerID $\rightarrow$ Customers.CustomerID
- Transactions.AccountID $\rightarrow$ Accounts.AccountID

Relationship Structure

- Customers (1) to Accounts (many)
- Accounts (1) to Transactions (many)

Visual Schema



Sample Data

Customers

	CustomerID	FullName	Email	Region	RegistrationDate
1		John Smith	john.smith@email.com	United States	2023-03-12

2	Maria Gonzalez	maria.gonzalez@email.com	Latin America	2023-06-25
3	Kwame Mensah	kwame.mensah@email.com	Africa	2023-08-03
4	Ana Silva	ana.silva@email.com	Latin America	2023-10-19
5	James Carter	james.carter@email.com	United States	2024-01-07
6	Sofia Ramirez	sofia.ramirez@email.com	Latin America	2024-02-14
7	Amina Diallo	amina.diallo@email.com	Africa	2024-04-02
8	Michael Brown	michael.brown@email.com	United States	2024-05-21
9	Lucia Torres	lucia.torres@email.com	Latin America	2024-07-09
10	Daniel Okafor	daniel.okafor@email.com	Africa	2024-09-16

Accounts

AccountID	CustomerID	AccountType	Balance	Currency
101	1	Checking	2450.75	USD
102	1	Savings	8200.00	USD
103	2	Checking	1350.20	MXN
104	3	Checking	980.00	GHS

105	4	Savings	5400.50	BRL
106	5	Checking	3120.10	USD
107	6	Checking	2250.00	MXN
108	7	Savings	4100.00	XOF
109	8	Savings	15000.00	USD
110	9	Checking	1750.75	BRL

Transactions

TransactionID	AccountID	TransactionType	Amount	TransactionDate
1001	101	Deposit	500.00	2025-11-10 09:15
1002	101	Purchase	42.35	2025-11-11 14:22
1003	102	Withdrawal	200.00	2025-11-12 10:05
1004	103	Deposit	750.00	2025-11-13 08:40
1005	104	Purchase	65.10	2025-11-14 16:18
1006	105	Withdrawal	300.00	2025-11-15 11:30
1007	106	Deposit	1200.00	2025-11-16 09:55

1008	107	Purchase	95.75	2025-11-17 18:06
1009	108	Deposit	400.00	2025-11-18 07:25
1010	109	Withdrawal	1000.00	2025-11-19 13:45

SQL Queries

Query 1 – Customer Account Count

```

57 • USE mydb;
58 • SELECT c.CustomerID, c.FullName, c.Region,
59       COUNT(a.AccountID) AS AccountCount
60 FROM Customers c
61 LEFT JOIN Accounts a ON a.CustomerID = c.CustomerID
62 GROUP BY c.CustomerID, c.FullName, c.Region
63 ORDER BY AccountCount DESC;
64

```

CustomerID	FullName	Region	AccountCount
1	Alice Johnson	Northeast	1
2	Bob Smith	Midwest	1

Result 17 × Read Only

Output

#	Time	Action	Message	Duration / Fetch
✓ 27	16:17:55	SHOW TABLES	3 row(s) returned	0.016 sec / 0.000 sec
✓ 28	16:19:35	SELECT * FROM Customers LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
✓ 29	16:20:16	SELECT * FROM Accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
✓ 30	16:21:08	SELECT * FROM Transactions LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
✓ 31	16:33:40	USE mydb	0 row(s) affected	0.000 sec
✓ 32	16:33:57	SELECT c.CustomerID, c.FullName, c.Region, ...	2 row(s) returned	0.000 sec / 0.000 sec

This query identifies how many accounts each customer holds, supporting customer segmentation and relationship management. It validates one-to-many relationship between Customer and Accounts in the data model.

Query 2: Total Balance by Region

The screenshot shows a SQL query editor with the following query:

```

64 • SELECT c.Region,
65       SUM(a.Balance) AS TotalRegionBalance
66 FROM Customers c
67 JOIN Accounts a ON a.CustomerID = c.CustomerID
68 GROUP BY c.Region
69 ORDER BY TotalRegionBalance DESC;
70

```

Below the query editor, the 'Result Grid' tab is active, displaying the following data:

Region	TotalRegionBalance
Midwest	7200.00
Northeast	3500.00

At the bottom, the 'Action Output' tab is visible, showing a log of database actions:

#	Time	Action	Message	Duration / Fetch
28	16:19:35	SELECT * FROM Customers LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
29	16:20:16	SELECT * FROM Accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
30	16:21:08	SELECT * FROM Transactions LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
31	16:33:40	USE mydb	0 row(s) affected	0.000 sec
32	16:33:57	SELECT c.CustomerID, c.FullName, c.Region, ...	2 row(s) returned	0.000 sec / 0.000 sec
33	16:35:59	SELECT c.Region, SUM(a.Balance) AS Total...	2 row(s) returned	0.031 sec / 0.000 sec

This query aggregates account balances by region to support geographic analysis and regional financial planning. It shows how the schema supports reporting using joins and aggregation.

Query 3: High-balance Accounts

Query 1 x

```

59      COUNT(a.AccountID) AS AccountCount
60  FROM Customers c
61  LEFT JOIN Accounts a ON a.CustomerID = c.CustomerID
62  GROUP BY c.CustomerID, c.FullName, c.Region
63  ORDER BY AccountCount DESC;
64  • SELECT c.Region,
65      SUM(a.Balance) AS TotalRegionBalance
66  FROM Customers c
67  JOIN Accounts a ON a.CustomerID = c.CustomerID
68  GROUP BY c.Region
69  ORDER BY TotalRegionBalance DESC;
70  • SELECT AccountID, CustomerID, AccountType, Balance, Currency
71  FROM Accounts
72  WHERE Balance >= 5000
73  ORDER BY Balance DESC;
74

```

Result Grid

	AccountID	CustomerID	AccountType	Balance	Currency
▶	102	2	Savings	7200.00	USD
*	NULL	NULL	NULL	NULL	NULL

Accounts 19 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 29	16:20:16	SELECT * FROM Accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
✓ 30	16:21:08	SELECT * FROM Transactions LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
✓ 31	16:33:40	USE mydb	0 row(s) affected	0.000 sec
✓ 32	16:33:57	SELECT c.CustomerID, c.FullName, c.Region, ...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 33	16:35:59	SELECT c.Region, SUM(a.Balance) AS Total...	2 row(s) returned	0.031 sec / 0.000 sec
✓ 34	16:36:58	SELECT AccountID, CustomerID, AccountType, ...	1 row(s) returned	0.000 sec / 0.000 sec

This query identifies high-value accounts for both risk-monitoring and targeted services.

Filtering and sorting support exception reporting.

Query 4: Transactions with Customer Name

```

74 • SELECT t.TransactionID, c.FullName, a.AccountType,
75       t.TransactionType, t.Amount, t.TransactionDate
76 FROM Transactions t
77 JOIN Accounts a ON t.AccountID = a.AccountID
78 JOIN Customers c ON a.CustomerID = c.CustomerID
79 ORDER BY t.TransactionDate DESC;
80

```

Result Grid

TransactionID	FullName	AccountType	TransactionType	Amount	TransactionDate
1002	Bob Smith	Savings	Withdrawal	500.00	2023-03-05
1001	Alice Johnson	Checking	Deposit	1500.00	2023-03-01

Result 20 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 30	16:21:08	SELECT * FROM Transactions LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
✓ 31	16:33:40	USE mydb	0 row(s) affected	0.000 sec
✓ 32	16:33:57	SELECT c.CustomerID, c.FullName, c.Region, ...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 33	16:35:59	SELECT c.Region, SUM(a.Balance) AS Total...	2 row(s) returned	0.031 sec / 0.000 sec
✓ 34	16:36:58	SELECT AccountID, CustomerID, AccountType, ...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 35	16:38:20	SELECT t.TransactionID, c.FullName, a.Account...	2 row(s) returned	0.000 sec / 0.000 sec

This query provides a full transaction audit trail by joining customers, accounts, and transactions.

It supports compliance, fraud monitoring, and customer service inquiries.

Query 5: Total Deposits vs Withdrawals per Account

```

80 • SELECT a.AccountID,
81       SUM(CASE WHEN t.TransactionType = 'Deposit' THEN t.Amount ELSE 0 END) AS TotalDeposits,
82       SUM(CASE WHEN t.TransactionType = 'Withdrawal' THEN t.Amount ELSE 0 END) AS TotalWithdrawals,
83 FROM Accounts a
84 LEFT JOIN Transactions t ON t.AccountID = a.AccountID
85 GROUP BY a.AccountID
86 ORDER BY a.AccountID;
87

```

Result Grid

AccountID	TotalDeposits	TotalWithdrawals
101	1500.00	0.00
102	0.00	500.00

Result 21 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 31	16:33:40	USE mydb	0 row(s) affected	0.000 sec
✓ 32	16:33:57	SELECT c.CustomerID, c.FullName, c.Region, ...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 33	16:35:59	SELECT c.Region, SUM(a.Balance) AS Total...	2 row(s) returned	0.031 sec / 0.000 sec
✓ 34	16:36:58	SELECT AccountID, CustomerID, AccountType, ...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 35	16:38:20	SELECT t.TransactionID, c.FullName, a.Account...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 36	16:39:27	SELECT a.AccountID, SUM(CASE WHEN t....	2 row(s) returned	0.000 sec / 0.000 sec

This query compares inflows and outflows per account, enabling cash-flow analysis supporting financial health monitoring.

Query 6: Transaction Count per Customer

```

87 • SELECT c.CustomerID, c.FullName,
88      COUNT(t.TransactionID) AS TransactionCount
89 FROM Customers c
90 JOIN Accounts a ON a.CustomerID = c.CustomerID
91 LEFT JOIN Transactions t ON t.AccountID = a.AccountID
92 GROUP BY c.CustomerID, c.FullName
93 ORDER BY TransactionCount DESC;
94

```

Result Grid

	CustomerID	FullName	TransactionCount
▶	1	Alice Johnson	1
	2	Bob Smith	1

Result 22 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 32	16:33:57	SELECT c.CustomerID, c.FullName, c.Region, ...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 33	16:35:59	SELECT c.Region, SUM(a.Balance) AS Total...	2 row(s) returned	0.031 sec / 0.000 sec
✓ 34	16:36:58	SELECT AccountID, CustomerID, AccountType, ...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 35	16:38:20	SELECT t.TransactionID, c.FullName, a.Account...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 36	16:39:27	SELECT a.AccountID, SUM(CASE WHEN t....	2 row(s) returned	0.000 sec / 0.000 sec
✓ 37	16:40:28	SELECT c.CustomerID, c.FullName, COUNT(...	2 row(s) returned	0.000 sec / 0.000 sec

This query identifies highly active customers, supporting behavioral analytics and fraud detection. It reinforces the relational integrity across all three tables.

Query 7: Largest Transaction (MAX)

```

100 • SELECT MIN(TransactionDate) AS EarliestTxn,
101      MAX(TransactionDate) AS LatestTxn
102 FROM Transactions;
103

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	EarliestTxn	LatestTxn
▶	2023-03-01	2023-03-05

Result 24 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 34	16:36:58	SELECT AccountID, CustomerID, AccountType, ...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 35	16:38:20	SELECT t.TransactionID, c.FullName, a.Account...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 36	16:39:27	SELECT a.AccountID, SUM(CASE WHEN t....	2 row(s) returned	0.000 sec / 0.000 sec
✓ 37	16:40:28	SELECT c.CustomerID, c.FullName, COUNT(...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 38	16:41:09	SELECT c.FullName, a.AccountID, t.Transaction...	1 row(s) returned	0.015 sec / 0.000 sec
✓ 39	16:42:12	SELECT MIN(TransactionDate) AS EarliestTxn, ...	1 row(s) returned	0.016 sec / 0.000 sec

This query identifies the largest transaction for risk and compliance review. It supports threshold-based monitoring commonly required in financial institutions.

Query 8: Earliest and Latest Transaction Date (MIN + MAX)

100 • `SELECT MIN(TransactionDate) AS EarliestTxn,`
 101 `MAX(TransactionDate) AS LatestTxn`
 102 `FROM Transactions;`
 103

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	EarliestTxn	LatestTxn
▶	2023-03-01	2023-03-05

Result 24 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 34	16:36:58	SELECT AccountID, CustomerID, AccountType, ...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 35	16:38:20	SELECT t.TransactionID, c.FullName, a.Account...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 36	16:39:27	SELECT a.AccountID, SUM(CASE WHEN t....	2 row(s) returned	0.000 sec / 0.000 sec
✓ 37	16:40:28	SELECT c.CustomerID, c.FullName, COUNT(...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 38	16:41:09	SELECT c.FullName, a.AccountID, t.Transaction...	1 row(s) returned	0.015 sec / 0.000 sec
✓ 39	16:42:12	SELECT MIN(TransactionDate) AS EarliestTxn, ...	1 row(s) returned	0.016 sec / 0.000 sec

This query identifies the operational transaction window of the system, supporting reporting timelines and system auditing.

Query 9: Transactions After a Date (WHERE + ORDER BY)

103 • `SELECT TransactionID, AccountID, TransactionType, Amount, TransactionDate`
 104 `FROM Transactions`
 105 `WHERE TransactionDate >= '2023-03-02'`
 106 `ORDER BY TransactionDate ASC;`
 107

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | **Result Grid** | Form Editor

	TransactionID	AccountID	TransactionType	Amount	TransactionDate
▶	1002	102	Withdrawal	500.00	2023-03-05
*	NULL	NULL	NULL	NULL	NULL

Transactions 25 x Apply

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 35	16:38:20	SELECT t.TransactionID, c.FullName, a.Account...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 36	16:39:27	SELECT a.AccountID, SUM(CASE WHEN t....	2 row(s) returned	0.000 sec / 0.000 sec
✓ 37	16:40:28	SELECT c.CustomerID, c.FullName, COUNT(...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 38	16:41:09	SELECT c.FullName, a.AccountID, t.Transaction...	1 row(s) returned	0.015 sec / 0.000 sec
✓ 39	16:42:12	SELECT MIN(TransactionDate) AS Earliest Txn, ...	1 row(s) returned	0.016 sec / 0.000 sec
✓ 40	16:43:57	SELECT TransactionID, AccountID, Transaction...	1 row(s) returned	0.015 sec / 0.000 sec

This query filters recent transaction activity to support operational monitoring and short-term trend analysis.

Query 10: Customers in a Region with Balances (JOIN + WHERE + ORDER BY)

```

107 • SELECT c.FullName, c.Region, a.AccountID, a.AccountType, a.Balance
108 FROM Customers c
109 JOIN Accounts a ON a.CustomerID = c.CustomerID
110 WHERE c.Region = 'Northeast'
111 ORDER BY a.Balance DESC;
112

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: I A

	FullName	Region	AccountID	AccountType	Balance
▶	Alice Johnson	Northeast	101	Checking	3500.00

Result 26 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 36	16:39:27	SELECT a.AccountID, SUM(CASE WHEN t....	2 row(s) returned	0.000 sec / 0.000 sec
✓ 37	16:40:28	SELECT c.CustomerID, c.FullName, COUNT(...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 38	16:41:09	SELECT c.FullName, a.AccountID, t.Transaction...	1 row(s) returned	0.015 sec / 0.000 sec
✓ 39	16:42:12	SELECT MIN(TransactionDate) AS Earliest Txn, ...	1 row(s) returned	0.016 sec / 0.000 sec
✓ 40	16:43:57	SELECT TransactionID, AccountID, Transaction...	1 row(s) returned	0.015 sec / 0.000 sec
✓ 41	16:45:02	SELECT c.FullName, c.Region, a.AccountID, a.A...	1 row(s) returned	0.000 sec / 0.000 sec

This query supports regional performance analysis by combining customer demographics with account balances. It enables targeted business strategies by geography.